Project 4: Decision Trees — TA Grading Rubric

Course: EN.601.464 Artificial Intelligence · **Assignment:** Project 4, *Decision Trees: from Entropy to Ensembles* · **Total: 200 points**

Students submit one `submission.zip` built by the notebook's `build_submission()` cell. It contains `project_4_decision_trees.ipynb` and a `recordings/` folder with 20 audio files, `manifest.json` (durations, timestamps, SHA-256 fingerprints) and `review.html` (one row per question with an inline player). Open `review.html` in a browser to listen; open the notebook to read code and outputs.

Component	Points	How it is graded
Code: autograded self-checks (21 tagged test cells)	100	`instructor/autograde.py` runs the notebook and scores each `test:` cell pass/fail
Code: TA review of code quality and experiments	10	Rubric §3.2
Voice answers (20 recordings)	70	Rubric §4, graded while listening to `review.html`
Process: Restart & Run All, docstrings, function length, complete submission	20	Rubric §5

The notebook is deliberately long (≈17 hours of student work across 8 parts). Expect submissions of 199 cells; the student-facing numbering (Part 0 … Part 8) matches the headings in the rubric.

---

1. What the assignment asks for (grader's map)

Part	Student builds	Deliberate problem the student must explain	Voice questions
0 Setup	runs the recorder, downloads the Kaggle data (253,680 BRFSS respondents, 13.93 % prevalence)	—	Q00 mic check
1 Entropy	`entropy_from_probabilities`, `class_probabilities`, `entropy`	#1 `math domain error` on a pure group (`0·log 0`)	Q01, Q02
2 Information gain	`partition`, `information_gain`, `split_info`, `gain_ratio`	#2 `patient_id` wins information gain (many-values bias)	Q03, Q04
3 ID3	`majority_class`, `best_attribute`, `id3`, `predict_one_naive`, `predict_one`, `classify`	#3 prediction returns `None` for an unseen attribute value	Q05, Q06, Q07
4 Real data & metrics	`confusion_counts`, `sensitivity`, `specificity`, `precision`, `f1_score`, `stratified_folds`	#4 every tree scores 86.07 % (accuracy paradox); duplicates	Q08, Q09, Q10

Part	Student builds	Deliberate problem the student must explain	Voice questions
5 Limits of one tree	`entropy_np`, `information_gain_np`, `best_threshold`, `should_stop`, `best_split`, `grow_tree`	#5 BMI split into 84 branches (fragmentation); #6 two halves give different trees (instability)	Q11, Q12, Q13, Q14
6 Ensembles	`bootstrap_indices`, `fit_bagging`, `predict_proba_bagging`, `predict_bagging`, `fit_adaboost`	#7 bagged trees share the same root (correlation)	Q15, Q16, Q17
7 Industry tools	scikit-learn tree, forest, gradient boosting (+ optional XGBoost), `undersample_balanced`, `threshold_for_sensitivity`	#8 precision collapses at real prevalence after balanced training	Q18, Q19 capstone
8 Submit	`lint_notebook_functions()`, `recording_status()`, `build_submission()`	—	—

---

2. How to grade voice answers

Listen with `recordings/review.html`. It lists every question with its prompt, the student's recording, its length and timestamp. 1.5× playback speed is fine; the rubric checklists are about content, not delivery.

Each question is scored on a checklist (§4). Award the listed points for each item the student states correctly and in their own words; partial items get half. Do not deduct for accent, hesitation, filler words, or reading from the student's own notes.

The student must engage with their own numbers. Nearly every prompt asks for a value from the student's outputs ("say the numbers you got", "quote your disagreement rate"). A correct general explanation with no numbers earns at most 60 % of the question's points; a number that does not match the notebook's output next to the question (within rounding) earns no credit for that item. The expected values for a correct implementation are in §7; a student with a different but correct-looking value whose notebook shows that value is fine.

Identity. Q00 is a mic check where the student states their name and JHED ID. Use it to confirm that the same voice answers all 20 questions. A clear voice mismatch across questions, or a recording that is obviously synthesized, goes to the instructor (do not grade it; note the question ids).

Script red flags. Reading a prepared answer is allowed; reading a generic essay is not what the question asks. Treat these as signs that the answer does not engage with the student's work, and cap the question at 50 %: no reference to any number or output from the notebook when the prompt asks for one; explanations that describe a behaviour the student's notebook does not show (e.g., "my tree picked BMI at the root" when the notebook shows `GenHlth`); verbatim textbook phrasing with none of the specifics of this dataset.

Length. Target lengths are guidance for the student. Do not deduct for a short answer that covers the checklist. Recordings under 10 seconds, silent, or unintelligible score 0 for that question; `build_submission()` warns the student about recordings shorter than the target, so these are rare.

Re-recordings and external recordings. Only the latest take is submitted. A recording flagged `EXTERNAL` in `review.html` / `manifest.json` was recorded outside the notebook (allowed fallback for microphone problems); grade it normally. A manifest whose SHA-256 does not match the file (`build_submission()` refuses to package such a submission, so this means manual tampering) goes to the instructor.

Integrity escalation. Escalate rather than penalise when: two students' recordings are near-identical in wording; a voice mismatch; a recording that answers a different question than asked in a way suggesting a shared answer bank.

---

3. Code rubric (110 points)

3.1 Autograded self-checks (100 points)

Run `python instructor/autograde.py submission.zip`. It executes the notebook (≈4–6 minutes) and prints PASS/FAIL per tagged test cell with the points below; the same checks are visible to students as assert cells. A test passes only if its cell raises no error. If a student's notebook fails to run top-to-bottom for a reason unrelated to the test (e.g., a renamed variable), the dependent tests fail; grade those by hand only if the fix is a one-line slip, and apply the Process deduction in §5.

Tag	Function(s) under test	Pts	What the cell checks	Partial credit (manual)
`entropy_from_probabilities`	`entropy_from_probabilities`	3	fair coin = 1 bit, 4 classes = 2 bits, 86/14 = 0.584	1 if formula right but wrong log base
`entropy_pure`	same, pure group	2	`[1.0, 0.0]` → 0.0 without error	0 if it still crashes
`entropy`	`class_probabilities`, `entropy`	3	fractions dict, empty list → 0, pure → 0, lecture labels 0.997	1 if only empty list fails
`entropy_real`	`entropy` on 253,680 labels	1	0.5825 bits	—
`information_gain`	`partition`, `information_gain`	6	partition sizes/order; gains 0.3845 / 0.2054 / 0.1635 / 0.2054 on the toy data	3 if partition correct but gain unweighted
`gain_ratio`	`split_info`, `gain_ratio`	4	split info 4.0 (16-way), 1.0 (8/8); ratios 0.2386 (id) and 0.2434 (BMI); single-value column → 0	2 if the zero-split-info case crashes
`best_attribute`	`majority_class`, `best_attribute`	4	majority ties → first seen; attribute ties → first candidate; works with `gain_ratio`	2 if ties broken differently

Tag	Function(s) under test	Pts	What the cell checks	Partial credit (manual)
`id3`	`id3`	10	reproduces TOY_TREE and the exact LECTURE_TREE; depth 0/1; counts on internal nodes; empty branch inherits parent majority	6 if trees right but counts/empty-branch missing; 3 if only depth-0/1 right
`predict_naive`	`predict_one_naive`	2	reproduces training labels on toy and lecture trees	—
`predict_one`	`predict_one`, `classify`	5	naive still returns `None`; fallback returns node majority at root and at an internal node; lecture error rates 0.0 and 7/15	3 if fallback uses the global majority instead of the node's
`metrics`	`confusion_counts`, `sensitivity`, `specificity`, `precision`, `f1_score`	6	hand-computed counts; zero-division cases return 0; string labels; all-negative predictor on the holdout	1 per correct metric
`stratified_folds`	`stratified_folds`	3	5 disjoint folds covering all indices, sizes within 2, prevalence within 0.5 %	1 if folds are random but not stratified
`entropy_np`	`entropy_np`, `information_gain_np`	4	matches the list versions on toy data and on a real column; empty/pure arrays	2 if gain right but entropy crashes on empty
`best_threshold`	`best_threshold`	6	perfect split at 3.5; `min_samples_leaf` makes it return `(None, -inf)`; constant column; dev-data BMI threshold is optimal by brute force	3 if midpoints wrong but best split found
`grow_tree`	`should_stop`, `best_split`, `grow_tree`	12	reproduces the Part 3 trees on encoded toy/lecture data; unseen value falls back and is flagged; real root `HighBP` then `GenHlth <= 2.5` both sides; `min_samples_leaf=500` respected; categorical mode equals ID3; `choose_pool` size	6 if categorical mode works but numeric splits fail; 3 if only stopping rules right

Tag	Function(s) under test	Pts	What the cell checks	Partial credit (manual)
`bagging`	`bootstrap_indices`, `fit_bagging`, `predict_proba_bagging`, `predict_bagging`	8	63.2 % distinct rows; list of `Node`s; vote fractions in k/7 steps; threshold 0 flags everyone	2 per function
`adaboost`	`fit_adaboost`	9	≥ 20 rounds; stumps; first error = prevalence; alpha formula; all alphas > 0 and errors in (0, 0.5); error rises in round 2	5 if weights update right but the stop rule is missing; 3 if resampling ignores the weights
`sk_tree`	scikit-learn tree	3	entropy criterion, depth 8, trained on all 202,944 rows, accuracy > 0.86, sensitivity > 0.10	—
`models`	`models` dict + `model_table`	4	required keys; RF with 100 trees; HGB; AUC RF > 0.80, HGB > 0.81, HGB ≥ tree	2 if one model missing
`undersample`	`undersample_balanced`	2	exactly 50 % positives, 2·(#positives) rows, no duplicates	—
`threshold`	`threshold_for_sensitivity`	3	hand example; every model reaches ≥ 0.80 sensitivity at the returned threshold and it is the largest such threshold	1 if sensitivity is reached but threshold not maximal

Hidden tests (optional): the function signatures are fixed, so staff may add extra asserts (e.g., `entropy` on a 3-class list, `id3` with `score=gain_ratio`, `stratified_folds` with `k=3`). The solution notebook passes all visible tests.

3.2 TA review of code quality and experiments (10 points)

Item	Pts	Full credit looks like
Functions do one thing and are not hard-coded to the toy data	3	no special cases for specific values; `id3` is genuinely recursive; no copying of expected outputs into return statements
Provided experiment cells produce their outputs (Part 4 depth sweep table, Part 5 learning-curve and `min_samples_leaf` plots, Part 6 bagging/forest/AdaBoost tables and plots, Part 7 ROC and threshold plots, feature importances)	4	all plots and tables are present after Restart & Run All; no cells skipped or commented out

Item	Pts	Full credit looks like
Part 7 model code follows the specification (`models` keys, hyper-parameters, `try/except ImportError` around XGBoost)	2	as specified in the task box; XGBoost absent is fine
No edits to course infrastructure or self-check cells	1	the infra cell and `test`-tagged cells are unchanged (diff against the distributed notebook if in doubt)

---

4. Voice question rubric (70 points)

For each question: points, target length, the checklist with per-item points, the model answer (numbers from the reference solution; the student's own notebook is the authority if its outputs differ because of a legitimate implementation choice), and answers that earn nothing. Items marked ⋆ are the causal explanation of a deliberate problem and carry the most weight.

Q00 · Mic check · 2 pts · 20–60 s

- 1 — states name and JHED ID clearly (identity reference for the other recordings).
- 1 — gives any prior conception of a decision tree (there is no wrong answer).
- Zero: silence, unintelligible audio, or a different student's voice.

Q01 · Entropy intuition · 3 pts · 45–90 s

- 1 — a 50/50 group has entropy 1 bit because one yes/no question resolves it (or: $-2 \cdot ½ \cdot \log_2 ½ = 1$); a pure group has 0 because there is nothing to resolve ($p \log p = 0$ for $p = 1$).
- 1 — a bit measures uncertainty / expected number of yes-no questions / information needed to name the label.
- 1 — quotes H ≈ **0.58 bits** for the diabetes label and interprets it: the problem is mostly predictable (14 % prevalence), so a question must be good to remove even a fraction of a bit.
- Model: "half/half means every patient is a coin flip, one question's worth of uncertainty; all-the-same means zero questions. Entropy counts expected yes/no questions. The real label came out at 0.58 bits, below one bit because 86 % say no."
- Zero: "entropy measures how random the tree is", or no number for the real label.

Q02 · ⋆ Deliberate problem #1, the pure group · 4 pts · 30–75 s

- 1 — describes the failure: `ValueError: math domain error` (or an equivalent crash) on `[1.0, 0.0]`.
- 1 ⋆ — cause: the formula multiplies 0 by $\log_2(0)$, and log of 0 is undefined ($\to -\infty$).
- 1 ⋆ — the limit of $p \cdot \log_2 p$ as $p \to 0^+$ is 0, so by convention $0 \cdot \log 0 = 0$ (p goes to zero faster than $\log p$ diverges).
- 1 — the fix: skip (or contribute 0 for) probabilities equal to 0; mentions that `entropy(labels)` built on `Counter` never sees zero counts, but the probability version must handle them.
- Zero: "I added a try/except", "I added a tiny epsilon" with no justification (epsilon gets 1 point if the limit is stated).

Q03 · Information gain by hand · 3 pts · 60–120 s

- 1 — parent entropy **0.954** bits (6 yes / 10 no).
- 1 — children: normal 5 patients, H = **0**; overweight 6 patients, H = **0.918**; obese 5 patients, H = **0.722**; weights 5/16, 6/16, 5/16; remainder = 0.344 + 0.226 = **0.570**; gain = **0.384** (accept ±0.01 and rounded values).
- 1 — why size-weighting: the remainder is the *expected* entropy after the question; a tiny pure group barely changes the expected uncertainty, a large mixed group dominates it; equal averaging would reward splits that peel off single patients.
- Zero: narrates the formula with no numbers from the toy data.

Q04 · ⋆ Deliberate problem #2, the identifier · 4 pts · 60–120 s

- 1 ⋆ — gain of `patient_id` = **0.954** = the parent entropy because every one of the 16 branches holds one patient and is pure, so the remainder is 0.
- 1 ⋆ — on the next patient the split is useless: a new id has no branch (the tree falls back / knows nothing); perfect training purity, zero generalisation; in general many-valued attributes are favoured because small groups are easy to make pure.
- 1 — gain ratio divides by split information (entropy of the branch sizes: 4 bits for 16 equal branches, 1.58 for BMI_class); in the student's run BMI_class **0.243** beat patient_id **0.239**.
- 1 — the margin is tiny, so gain ratio reduces but does not remove the bias; mentions a real remedy (C4.5's average-gain guard, binary splits as in Part 5, or simply never feeding ids to the learner).
- Zero: "patient_id is correlated with diabetes".

Q05 · ID3 stopping rules · 3 pts · 60–120 s

- 1 — all four stops: pure node; no candidate attributes left; depth limit reached; empty data for a branch value.
- 1 — leaf labels: majority label of the node's records in the first three cases (the pure case trivially); the **parent's** majority for an empty branch.
- 1 — why the parent's majority: the empty branch has no evidence of its own; the parent summarises every training patient who reached that point, so it is the most informed guess; it is the same fallback `predict_one` uses at prediction time.
- Zero: omits the empty-branch case, or says an empty branch returns `None`/raises.

Q06 · ⋆ Deliberate problem #3, unseen value · 4 pts · 45–90 s

- 1 ⋆ — where it got stuck: the root tests `BMI_class`, the patient's value `underweight` has no child, so the walk has nowhere to go and the naive function returns `None`.
- 1 — consequence: `None` propagates into scoring (`error_rate` raised / counted wrong); in production a crash mid-clinic; this happens constantly on real data (84 BMI values; ≈8 % of holdout patients hit an unseen branch in Part 5).
- 1 — the fix: return the current node's stored majority (`prediction`) when the child is missing; quotes that the root majority is `no` (10 of 16) or the obese node gives `yes`.
- 1 — why sensible: the node's majority is conditioned on every earlier answer the patient gave; worse alternatives: crashing, a random label, always `no`/always `yes`, or the global majority that ignores the path.
- Zero: "I added `underweight` to the training data".

Q07 · Greedy choice · 2 pts · 30–60 s

- 1 — ID3 is greedy and conditional: after the root split on `BMI_class`, the best attribute is re-evaluated *within each child*; inside the overweight group `Age_group` separates perfectly (gain 0.918 vs 0.459 for `HighBP`), inside the obese group `PhysActive` does.
- 1 — therefore a globally second-best feature can be redundant given the first split and never be chosen; the algorithm never looks back (no global optimisation).
- Zero: "HighBP had low gain".

Q08 · ⋆ Deliberate problem #4, the 86 % wall · 4 pts · 60–120 s

- 1 ⋆ — prevalence is 13.9 %, so predicting "no diabetes" for everyone is right **86.07 %** of the time; depth 0 is exactly that model.
- 1 ⋆ — a leaf's prediction only changes when it holds **more than 50 % diabetics**; the first splits lower entropy (gain is real) but every child is still majority-no, so accuracy is unchanged; when leaves finally flip, holdout accuracy goes *down* (depth 4: ≈0.846, depth 6: ≈0.818) while training accuracy rises.
- 1 — quotes sensitivity **0.0** at depth 0 and ≈**0.25** at the deepest tree (depth 6 pure-Python: 0.245).
- 1 — verdict: 86 % accuracy is useless as a screen, because it finds no diabetic patient; accuracy hides the minority class.
- Zero: "my code was broken", or "86 % is a good model".

Q09 · Sensitivity vs specificity · 3 pts · 60–120 s

- 1 — definitions from the confusion matrix: sensitivity = TP / (TP + FN), specificity = TN / (TN + FP); correctly names which cell is the missed patient (FN) and the unnecessary test (FP).
- 1 — quotes numbers for one of their trees (e.g., depth 6: sensitivity ≈ 0.245, specificity ≈ 0.911; depth 2: 0.047 / 0.994).
- 1 — screening: sensitivity matters more (a miss is a lost diagnosis; a false alarm costs a confirmatory blood test); confirmatory test before medication: specificity/precision matter more (treating a healthy person has side effects).
- Zero: swaps the two definitions.

Q10 · Duplicates and leakage · 2 pts · 30–75 s

- 1 — a duplicate in both folds means the model is tested on a row it memorised, so the estimate is optimistic; yes, it is a form of leakage (the test fold is not independent of the training fold); with 14 % duplicated rows the effect is small but systematic.
- 1 — remedy: deduplicate before splitting, or keep identical rows in the same fold (group-aware folds); notes the nuance that identical rows may be different people with identical coarse answers, which is why coarse surveys produce them.
- Zero: "duplicates make the model more accurate, which is good".

Q11 · Learning curves · 3 pts · 60–120 s

- 1 — shape: training error decreases monotonically with depth (to ≈0.09 at full depth); holdout error decreases, bottoms out, then rises above the predict-nothing line (≈0.139).
- 1 — quotes the depth at the holdout minimum in their run (reference: actual depth **5–8**, holdout error ≈ 0.136–0.137).

- 1 — training error keeps falling because a deeper tree can memorise its 50,000 rows (eventually pure leaves); the holdout turns around because the extra splits fit sample noise, not the population; the name is **overfitting** (variance).
- Zero: calls the right side "underfitting".

Q12 · ⋆ Deliberate problem #5, fragmentation · 4 pts · 45–90 s

- 1 — quotes leaves: categorical ID3 ≈ **17,000** vs threshold tree ≈ **8,700**; unseen-branch rate ≈ **8 %** vs **0 %**.
- 1 ⋆ — why one-branch-per-BMI-value splits (68 and 58 branches under GenHlth 4 and 5) hurt: each branch holds a handful of patients, so its "pure" label is noise (Part 2's identifier problem in disguise); training accuracy rises because fragmentation memorises, but holdout sensitivity/F1 are lower (F1 ≈ 0.28 vs 0.31) and many patients fall on branches that do not exist.
- 1 ⋆ — threshold splits ask `BMI <= t`, a two-way question with statistical power on both sides, and can revisit BMI deeper with another cut; this is how CART/scikit-learn avoid the many-values bias.
- 1 — links back to Part 2 (gain's bias toward many values / gain ratio) or notes that the fallback silently handled 8 % of predictions.
- Zero: "the threshold tree is better because it is smaller" with no mechanism.

Q13 · ⋆ Deliberate problem #6, instability · 4 pts · 60–120 s

- 1 — compares the roots: tree A `GenHlth <= 2.5`, tree B `HighBP` (or whatever the student's run shows; must match their printout).
- 1 — quotes the disagreement rate ≈ **14.6 %** of holdout patients while both trees have similar accuracy (≈0.83).
- 1 ⋆ — why high variance: the greedy root choice hinges on small gain differences; once the root differs every subtree differs; full-depth trees then fit the noise of their particular sample.
- 1 — the physician's concern: a tool whose rules change when retrained on new data is hard to trust, audit, or explain to patients; predictions for the same person flip; motivates averaging many trees (Part 6).
- Zero: "the two halves had different patients so of course the trees differ" with no mechanism.

Q14 · Income and Education in a clinical tool · 3 pts · 45–90 s

- 1 — one argument for: these features carry real predictive signal (access to care, diet, environment); dropping them can lower sensitivity for the very groups most at risk.
- 1 — one argument against: they are proxies for protected or sensitive attributes, can encode and perpetuate disparities, are not clinically actionable, and may be legally or ethically unacceptable as a basis for triage.
- 1 — a position plus what to check: subgroup sensitivity/specificity and calibration across income/education groups, whether the features add anything beyond clinical ones (ablation), stakeholder/IRB review, and how the output is used (prioritising outreach vs denying care).
- Zero: only one side, or no position.

Q15 · Bagging · 3 pts · 60–120 s

- 1 — bootstrap aggregation: draw n rows with replacement (≈63 % distinct), grow an unpruned tree on each, average/vote; averaging reduces variance, not bias.

- 1 — quotes the student's numbers at threshold 0.5: accuracy **up** (≈0.832 → 0.859), sensitivity **down** (≈0.28 → 0.17), F1 down (≈0.31 → 0.26); explains that a minority-class patient rarely gets a majority of votes.
- 1 — lowering the threshold (e.g., 0.2) raised sensitivity to ≈0.62 at specificity ≈0.80 (F1 ≈ 0.44); the vote fraction is a probability-like score that can be thresholded, whereas a single tree outputs one hard label per leaf.
- Zero: "bagging always improves every metric".

Q16 · ⋆ Deliberate problem #7, correlated trees · 4 pts · 45–90 s

- 1 — quotes the root counts: ≈ **17 of 25** bagged trees rooted at `HighBP`, the rest at `GenHlth`.
- 1 ⋆ — why correlated: bootstrap samples share most of their rows and the strongest feature wins the root almost every time; averaging correlated estimators leaves a variance floor (the trees make the same mistakes).
- 1 ⋆ — `max_features` restricts each node to a random subset of columns, forcing trees to use second-best questions and decorrelating them.
- 1 — reports the change: roots spread over many features (GenHlth, HighChol, Age, DiffWalk, HighBP, …); AUC and F1 at threshold 0.2 edge up (≈0.44 → 0.45); notes the gain is modest here because a few features dominate.
- Zero: "max_features makes the trees faster" as the only explanation.

Q17 · AdaBoost · 3 pts · 60–120 s

- 1 — weights: misclassified rows are multiplied by e^α, correctly classified by e^−α, then renormalised (equivalently: the two groups end up with equal total weight).
- 1 — round 1: with uniform weights the best stump predicts "no" for everyone, so its weighted error equals the prevalence ≈ **0.14**; α = ½·ln(0.86/0.14) ≈ **0.91**, large because the stump is far better than chance *on that distribution*.
- 1 — the reweighting multiplies every diabetic patient's weight by ≈2.5 and shrinks the others, so by round 2 the data is nearly balanced and stumps must find features that separate diabetics (GenHlth, BMI, Age…); this is how boosting attends to the minority class a single tree ignored.
- Zero: "boosting trains the trees in parallel".

Q18 · ⋆ Deliberate problem #8, base rates · 4 pts · 45–90 s

- 1 — sensitivity and specificity are properties of the test, precision depends on prevalence (Bayes / base rates).
- 1 ⋆ — the arithmetic per 1,000 screened at 14 % prevalence: ≈139 diabetic × 0.79 ≈ **110 true positives**; ≈861 healthy × 0.29 ≈ **250 false positives**; precision ≈ 0.30 (accept 100–120 TP, 230–270 FP).
- 1 ⋆ — on a balanced test set the same test gives ≈395 TP vs ≈145 FP (precision ≈ 0.73): nothing about the model changed, the denominator did.
- 1 — conclusion: a balanced test set lies about precision; evaluate (and set the threshold) at deployment prevalence; balanced training is only a way to move the operating point.
- Zero: "the model overfit the balanced data".

Q19 · Capstone recommendation · 8 pts · 120–180 s

- 2 — a specific recommendation with numbers: a model (gradient boosting or random forest is expected; a well-argued tree is acceptable) and an operating threshold, quoting sensitivity, specificity and precision at that threshold (reference for gradient boosting at 80 % sensitivity: threshold ≈ 0.13, specificity ≈ 0.70, precision ≈ 0.30).
- 2 — two concrete failure modes, e.g.: self-reported survey features differ from clinic data (distribution shift); 2015 US population vs the deployment population; calibration drift; subgroup performance gaps; unseen/missing values; duplicates inflating validation numbers; false positives overwhelming confirmatory testing capacity.
- 1 — one ethical or practical concern: use of income/education, burden of false positives on patients, missed cases, consent for using survey-derived risk, automation bias.
- 2 — what to monitor after deployment: prevalence and positive rate, sensitivity/specificity on confirmed outcomes, calibration, subgroup metrics, data drift, threshold revisits.
- 1 — the reflection: one sentence contrasting their Q00 answer with what they now understand (any honest contrast earns the point).
- Zero: a recommendation with no numbers, or "accuracy 86 %" as the justification.

---

5. Submission checks and penalties (20 points)

Check	Pts	How to verify	Deduction guidance
Restart & Run All: notebook runs top to bottom without error, cells numbered in order	6	execution counts are 1, 2, 3, … in order; `autograde.py` reports no non-test cell errors	−6 if cells are out of order or any cell errored; −3 if a single cosmetic cell failed (e.g., XGBoost import without try/except)
Docstrings on every function the student wrote	4	`lint_notebook_functions()` output in Part 8 (`has_docstring` column)	−1 per missing docstring, floor 0
Functions about 20 lines or fewer (earnest attempt)	3	same table, `body_lines` column	−1 per function over 30 lines; ignore 21–30
Complete submission: `submission.zip` built by the notebook, containing the notebook, 20 recordings, `manifest.json` and `review.html` ; manifest hashes match the files	4	unzip and open `review.html` ; `autograde.py` prints the manifest summary	−4 if the zip was assembled by hand with recordings missing or renamed; −2 if 1–2 recordings missing (and 0 for those questions in §4)
Recordings meet the target length (within reason) and are audible	3	`review.html` shows each duration	−1 per recording shorter than half its target or inaudible, floor 0

Other penalties (apply once each): submitting a PDF or an `.ipynb` outside the zip (−5, and grade from the notebook if recordings were uploaded separately); data files or the kagglehub cache inside the zip (−2); modified infrastructure or test cells (−5 and grade the autograded part by re-running with the original cells).

---

6. TA workflow and time budget

1. **Batch the autograder.** `for z in submissions/*.zip; do python instructor/autograde.py "$z" --json "${z%.zip}.json"; done` Each run is 4–6 minutes on a laptop (the bagging cell dominates); run them in parallel or overnight. Record the 100-point code score.
2. **Open the notebook** (the executed copy produced by the autograder is fine) and score §3.2 (10 pts) and §5 (20 pts): about 5 minutes.
3. **Open** `recordings/review.html` and grade §4 top to bottom with the checklists; type the item points as you go. Total audio per student is 15–25 minutes; at 1.5× speed, plus note-taking, budget **20 minutes per student**. Grade one question across several students in a row if you prefer consistency over speed.
4. **Escalate**, do not resolve: voice mismatches, manifest hash mismatches, near-identical recordings across students, and any suspected use of synthesized speech. Note the submission and question ids for the instructor.
5. **Feedback.** Gradescope comments should name the checklist item missed (e.g., "Q08: did not state what a leaf must contain before its prediction flips"). Students see the rubric item names in this document.

Time per submission: ≈30 minutes of TA attention plus unattended autograder time. For a section of 100 students that is about 50 TA-hours; plan a grading rota by Part rather than by student if several TAs share the load, so that each Part is graded consistently.

---

7. Reference values from the solution notebook

All experiments are deterministic (seeds fixed; dev sample = 50,000 stratified rows of the 80 % training split; holdout = 50,736 rows). Student values should match these to the printed precision unless their implementation makes a different legitimate choice (e.g., a different tie-break), in which case their notebook output is the reference.

Quantity	Value
Dataset	253,680 × 21 features; prevalence 0.1393; 24,206 exact duplicate rows (35,575 rows, 14.0 %, have a twin)
Entropy of the label	0.5825 bits
Toy data	H = 0.9544; gains: BMI_class 0.3845, HighBP 0.2054, PhysActive 0.2054, Age_group 0.1635; with id: gain 0.9544, gain ratios id 0.2386 vs BMI 0.2434
Part 4 pure-Python ID3 on 50k dev, holdout accuracy by depth 0–6	0.8607, 0.8607, 0.8619, 0.8581, 0.8464, 0.8293, 0.8184 (sensitivity 0, 0, 0.047, 0.092, 0.156, 0.212, 0.245)
Part 5 threshold tree depth-2	root `HighBP`, then `GenHlth <= 2.5` on both sides
Learning curve (min_samples_leaf 5)	holdout error minimum ≈ 0.136 at actual depth 5–8; full depth: train error ≈ 0.09, holdout ≈ 0.17

Quantity	Value
Fragmentation	categorical: 17,079 leaves, depth 15, unseen-branch rate 8.2 %, holdout acc 0.803, F1 0.280; threshold: 8,719 leaves, depth 38, 0 % unseen, acc 0.799, F1 0.308
min_samples_leaf sweep (holdout)	accuracy peaks ≈ 0.865 at m ≈ 100; F1 peaks ≈ 0.31 at m = 5
Instability	roots `GenHlth <= 2.5` vs `HighBP`; disagreement 14.6 %; accuracies 0.832 / 0.828
Bagging, 25 trees, threshold 0.5	acc 0.859, sens 0.174, F1 0.256 (single tree 0.832 / 0.276 / 0.314); at 0.2: sens 0.62, spec 0.80, F1 0.436; roots HighBP 17, GenHlth 8
Random forest, max_features 4	roots spread over ≈9 features; threshold 0.2: F1 ≈ 0.445, balanced accuracy ≈ 0.70
AdaBoost, 50 stumps	errors 0.139, 0.31, 0.35, 0.42 …; alpha_1 0.91; 50 rounds: acc 0.865, sens 0.19, F1 0.28
scikit-learn (full training set)	tree depth 8: acc 0.864, sens 0.136, AUC 0.815; RF 100/msl 20: AUC 0.825; HistGradientBoosting: acc 0.866, AUC 0.828; XGBoost: AUC 0.829
Balanced training (HGB)	real holdout: sens 0.794, spec 0.707, precision 0.305; balanced holdout: precision 0.735
80 % sensitivity operating point	HGB threshold ≈ 0.133, specificity ≈ 0.70, precision ≈ 0.30; RF ≈ 0.69 / 0.30; tree ≈ 0.67 / 0.28
Feature importances (RF)	GenHlth 0.18, BMI 0.18, HighBP 0.15, Age 0.09, HighChol 0.08, DiffWalk 0.07